

Randomised Binary Search Trees:

A Complexity and Performance Analysis

Adam Kearns⁽²³³²⁶⁹⁰³⁾, Cormac Kehoe⁽²³⁴¹¹³⁰²⁾, William Lin⁽²³³¹⁴²¹³⁾, Liam Maher⁽²³⁴⁴⁴⁸⁸²⁾

University College Dublin, Dublin, Ireland



I. Introduction to Treaps:

A Treap is a hybrid data structure built using a combination of the properties of a Heap and a Binary Search Tree ^{abbrv. BST}.

Fundamentally a Treap is built of nodes that contain three core pieces of data: its Key (used to maintain its BST properties), its Priority (used to maintain its Heap properties) and its Value.

The BST property of the node is that for any node N , all nodes in its left subtree have Key value $< N.key$ and all nodes in its right subtree have Key value $> N.key$ while the Heap property states that $N.Priority$ is $\leq parent(N).Priority$. The Treap is balanced probabilistically via the random assignment of priorities, resulting in an expected $O(h)$ time complexity for insertion, deletion and search operations, where h is the height of the tree. Due to its probabilistic balancing, this means the height of the tree is expected to be on average, $\log n$, meaning $O(\log n)$.

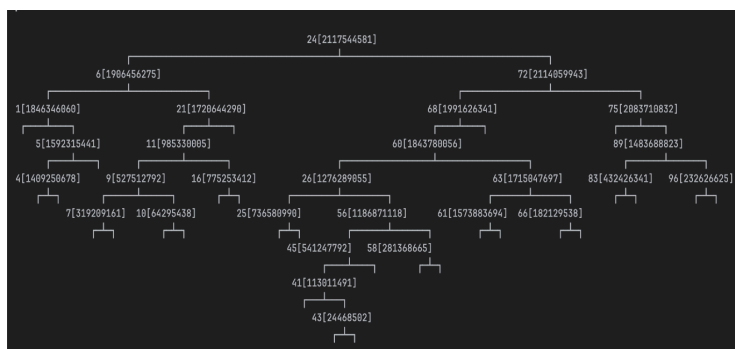
For the insertion of nodes in a Treap we firstly insert the node using the key just as in a BST. This means the node is inserted as a leaf node. Then, to maintain the heap property of the treap, we must upheap the node until its priority satisfies the heap. These upheap operations are done using tree rotations, which are constant time operations.

For the deletion of nodes, we first check if the node has any children, if it has none, it is a leaf node and we can simply unlink it from its parent which removes it from the tree. If it's not a leaf, we must perform tree rotations down-heaping the node until it becomes a leaf. These rotations maintain the heap property and BST property of the treap, once we have a leaf node, we again unlink it from its parent node.

Searching the Treap is done identically to a BST (we compare the keys and then go either left or right).

Here is an example of the output of a treap with each node consisting of **KEY** [**PRIORITY**].

Note that the priority is normally hidden from the user, just for visualisation



II. Design Decision:

Our general implementation of the treap involved extending the treemap class, and overriding necessary methods to adjust for treap functionality, notably `rebalanceInsert`, for rebalancing the tree after an insertion operation, and the `remove` method, which has different logic to treemaps method completely.

Adam:

[Adams Repo](#)

My implementation of treap consists of the 2 separate classes, `Treap`, which extends `treemap`. And `TreapBinaryTree`, which extends `BalanceableBinaryTree`. My `TreapNode` class is a nested class in the `TreapBinaryTree`. Something that I noticed I implemented differently was the way I assign the priority values to the nodes. Instead of using `setPriority`, I assign the nodes a priority randomly in the constructor.

- For my `'rebalanceInsert()'`, this overrides the treemaps method. Calling `put` simply uses the super classes method, to insert the node using BST logic, then to restore the heap property, the method involves checking the inserted node's priority against its parents', and bubbling the node up using tree rotations until the heap property is restored.
- For my `'remove()'`, I implemented it in a way that still uses the `TreeMap` remove logic. First, if the node is a leaf I just remove it by unlinking it from its parent. Or else I call the `'downheap()'` method, which uses tree rotations to sink the node down the tree until it becomes a leaf, at which point it is removed from the tree again using `TreeMaps` remove method.
- For my checking methods, they are both recursive functions that start at the root of the tree, and check if the heap and BST properties are satisfied for the root, then I recurse down the left and right subtrees and finally the method returns a boolean if the respective properties are valid, there's also a combination of the 2, `'isValidTreap()'`, which simply runs both recursive functions.

I also made a set of simple unit tests for each of the above methods, csv files for the more in depth benchmarking are also included in the project, these were run using the 'Main' class. Running again will override current results

Will:

[Williams Repo](#)

My TreapBinaryTree extended BalanceableBinaryTree consisted of the node class for treap which included a priority that was randomly allocated using the random utility when method createNode is used. This was crucial to enable the nodes to do rotations and structural modifications needed by treaps. My balancing and logic of the actual treap was contained in my Treap class. My Treap extended TreeMap which allows me to use the tree logic to my treap and also giving it the heap behaviour. I overriden the rebalanceInsert() and rebalanceDelete() from treemap which were used to also call upHeap and downHeap which performed rotations to keep the heap property. These rotations are performed here to move higher or lower the priority nodes to be in line with the heap property. I used the inherited put and get methods which called my overriden rebalance methods which would check the conditions for the heap and tree were met. To check these conditions my code also includes a checkBST and checkHeap which are both used to check the validity of the treap. This ended up being my implementation of Treap after receiving the tests that were made to test these and I had to refactor my original design of Treap but it has made my treap handle some more exceptions/errors than it did.

Liam:

[Liams Repo](#)

My Treap implementation in Java extends an existing 'TreeMap' framework with a custom 'TreapBinaryTree' that generates and stores priorities for each node. Key design decisions include:

- Extending 'BalanceableBinaryTree' to reuse existing rotation logic.
- Creating a nested 'TreapNode' class that includes a 'priority' field.
- Using Java's 'Random' class to assign priorities upon node creation.
- Maintaining the heap property via 'rebalanceInsert' (upheap) and 'downheap' during deletions. These upheap and downheap methods were implemented using the tree rotations from the extension of the 'BalanceableBinaryTree'.
- Validating both BST and heap properties with recursive 'checkBST()' and 'checkHeap()' methods.

This design made it easier to integrate with the benchmarking framework while maintaining clean separation of logic.

Cormac:

[Cormacs Repo](#)

My Treap implementation is split into two separate classes, *Treap* and *TreapBinaryTree*. By doing this Treap benefits from the existing functionality of Treemap (*put()*, *remove()* etc.).

The TreapBinaryTree class contains the subclass defining a TreapNode, which is a variation on the standard BinaryTreeNode that also contains a randomly assigned Priority. I opted to assign the priority randomly upon node creation without a setter method, rather assigning the values within the constructor itself. This creates it such that nodes have a priority immediately upon construction.

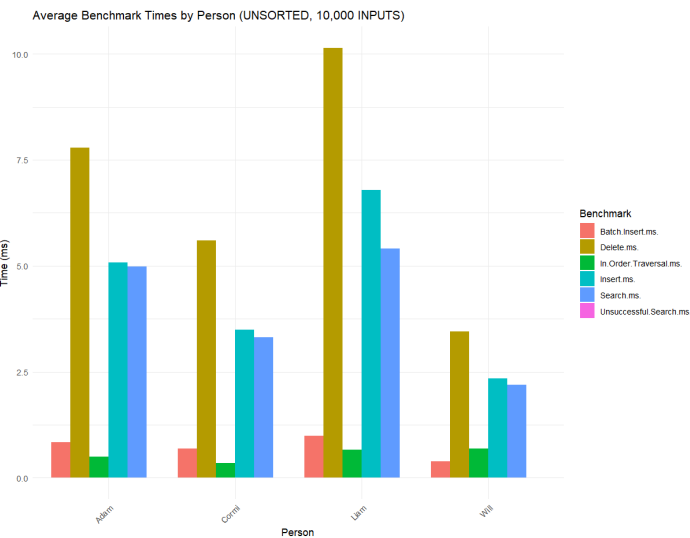
By using a custom TreapBinaryTree, I was able to handle priorities and rebalancing in an encapsulated manner. This separation provided more flexibility for implementing the Treap behaviour.

By utilising rotations I was able to maintain the Binary Search Tree and Heap properties after each insertion or removal from the tree. This was achieved by rotating a node either to or from being a leaf node upwards or downwards through the tree.

I opted to create two separate helper methods to implement *isValidTreap()*. This permitted me to test for both Binary Search Tree and Heap properties independently.

As for benchmarking and testing, I opted for a given, when, then structure to clarify exactly what changes in each test. The data from each of these tests was printed into separate CSVs. There are pdfs of the results displayed in a Google Sheet in the Git. (This is the raw data without any average, rounding, or grouping done to it).

III. Performance Results:



This graph shows the results of running a series of benchmarking tests on each of our individual implementations of the treap. They are ran on an unsorted treap with 10000 inputs. As is clearly shown, the general trend of each of the graphs remains the same. One of the key takeaways from this is that deletion operations take the most time to execute. This is due to the fact that for a deletion of the node, it first must be checked if the node is in the treap, and then further having to sink the node into the treap before it is removed, whereas for an insertion, we don't have to check if the key exists in the code or not, we simply just insert as a leaf and bubble the node up until the priority heap property is restored. The times for insertion and search operations are expected to have $O(\log n)$ time complexity and as evident by the graph this holds, as the 2 columns are relatively similar for most of the implementations. The differences between timings are a result of the hardware of each of our devices more than an implementation detail.

IV. Sorting Algorithms:

Benchmarks were conducted by comparing TreapSort against PQSort, JavaSort, QuickSort and MergeSort across input sizes of 10, 100, 100, 1000 and 10000 elements, and results very clearly indicate that TreapSort is slower across the board:

Cut out from the full sorting results uploaded in submission

Input type	Input size	TreapSort	PQSort	JavaSort	QuickSort	MergeSort
Random	100	2.73828	0.11108	0.11918	0.08682	0.07266
Random	1000	1.19993	0.19274	0.23675	0.15092	0.22454

Analysis:

Each sort maintains an average $O(n \log n)$ time complexity, but performance differs significantly:

- **QuickSort** - Excellent average-case performance and low overhead, ideal for general use.
- **MergeSort** - Stable sort, great for linked lists or when stability is required.
- **PQSort** - Relies on heap structure, slower than Merge/Quick but useful when priority-based sorting is required.
- **JavaSort** - Hybrid and highly optimized, performs best in most real-world cases.
- **TreapSort** - Theoretically sound but practically slower due to extra balancing overhead.

The array-based sorting algorithms (JavaSort, QuickSort, MergeSort) benefit from tight memory locality and few object allocations unlike TreapSort which suffers some significant object overheads, pointer chasing and randomisation costs. And although TreapSort maintains an average-case $O(n \log n)$ complexity, the constant factors make it slower in practice.

V. Treap vs AVLTreeMap:

While AVL trees offer strictly balanced structures and a consistent $O(\log n)$ performance, Treaps hold the advantage in the following ways:

- Randomised behaviour, which avoids worst case inputs that can degrade AVL trees if they are not balanced properly.
- Simplicity of the balancing logic. By using tree rotations and priority comparisons instead of height updates, the implementation of the logic is a lot more straightforward for treaps.
- For applications where tree insertions and deletions are much more frequent and a balanced tree is a less critical component, treaps excel over AVL trees due to the probabilistic balancing based on their random node priority.

In summary treaps are the more efficient solution if you want a randomised, self balancing structure with simpler code, you don't need strict balancing guarantees or if performance predictability is less important than simplicity and average-case performance